

Artificial Intelligence (CS-203)

Objectives:

- Pattern matching
 - ELIZA
-

Pattern Matching

Matching is a very useful technique, which is used by human being to solve problems, to do recognition, and to do learning. Human beings are excellent pattern matcher, computers are far behind in this field. Now a days AI systems are also using this technique to solve the problems more efficiently. There are three types of matching.

1. Image matching
2. Sound matching
3. Symbol matching

1. Image matching

Image matching is active area of research. There are many monitoring systems and security systems, which take visual input. It is a very vast field of AI applications. Example of system being used by BOING (Air craft manufacturing company) for quality controlling of construction of tubes and pipes. A Laser beam is passed on the pipe to get its 3-D image then that image is matched against the 100% correct images of pipes, then accepted or rejected accordingly.

2. Sound matching

Many advance systems have voice input and output. For this they have sound matcher module to recognize and understand the voice of the user. It is also an active area of research.

3. Symbolic matching

This is the easier type of matching in which we match different symbols or line of text with each other. These symbols are usually representing some situation or state. To find the next step or solution to the situation system match these symbols with those already stored in the system. Symbolic matching can be further categorizing into three types, Matching in Game playing, Matching with variables, Complex and approximate matching.

i. Matching in Game playing

Current state of the board is matched with the preconditions of rules, so that next move is selected. Such game playing systems have each move in the form of rule (if__ then__ construct). When a rule should be executed (move should be made) depends upon the situation of the board. System matches the situation of the board with the preconditions of rules and selects all those rules (moves) which can be made in the current situation.

ii. Matching with variables

Prolog is based on pattern matching. In Prolog rules are indexed by predicates, so all the rules that could be applicable to proving a particular fact can be accessed fairly quickly. Input query sentence (containing the variables) is matched with the facts existing in the knowledge base. A Logical variable can match any symbol.

iii. Complex and approximate matching

Natural Language Processing Systems use more advance technique of symbolic pattern matching. Voice recognition systems use approximate matching algorithms. Approximate matching is more difficult because the tolerance factor may lead to wrong matching.

Introduction of Eliza

Eliza was one of the first programs to feature English output as well as input. The program was named after the heroine of a Pygmalion, who was taught to speak proper English by a dedicated teacher. Eliza's principal developer, MIT professor Joseph Weizenbaum, published a paper on Eliza in the January 1966, issue of the communications of the association for computing machinery. Eliza is a program which behaves like a psychoanalyst. It talks to the user, let the user say its problem. In Eliza every sentence of discussion is already programmed in the form of rule.

Example	Stored Rules
Hello my name is Ali Ahmed	1. "?*x my name is ?*y": ["How are you ?y?", "How can I help you ?y?"]
My name is Kashif Jamil	
Hi my name is Anwer Ali	2. "?*x I need ?*y": ["What would it mean if you got ?y?", "Why do you want ?y?", "Suppose you got ?y soon."]

These will match first rule, response could be

HOW ARE YOU <NAME>

Again user says any one of the following

I am thirsty, I need a glass of water

SUPPOSE YOU GOT A GLASS OF WATER

Actually I need help in AI

WHY DO YOU NEED HELP IN AI

Pattern Matching Modules

Logical Variable and Binding List;

In pattern matching module we shall discuss a logical variable a special symbol which starts with '?'. It can match with exactly one symbol. So if we match the following user input with rule pattern;

Input:

'?a my name is ?n'

→ where ?a and ?n are logical variables

'Hello my name is Ali'

Output:

{'a': ['hello'], 'n': ['ali']}

→ Binding List

1) is_variable function:

To take a symbol and tell whether it is a logical variable or not.

```
letters = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"
```

```
def is_variable(pattern):
    """Test if pattern is a single variable."""
    return (type(pattern) is str
            and pattern[0] == '?'
            and len(pattern) > 1
            and pattern[1] != '*'
            and pattern[1] in letters
            and ' ' not in pattern)

>>> is_variable("?a")
True
>>> is_variable('?a')
True
>>> is_variable('?abc')
True
```

```
>>> is_variable('?foo')
True
>>> is_variable('foo')
False
```

2) match_variable function:

takes a variable and its value (replacement) and stores it in binding dictionary

```
def match_variable(var, replacement, bindings):
    """Bind the input to the variable and update the bindings."""
    binding = bindings.get(var)
    if not binding:
        # The variable isn't yet bound.
        bindings.update({var: replacement})
        return bindings
    if replacement == bindings[var]:
        # The variable is already bound to that input.
        return bindings
    # The variable is already bound, but not to that input--fail.
    return False
```

```
Bindings = {}
>>> match_variable("?a", 10, bindings)
{'?a': 10}
>>> match_variable("?b", 20, bindings)
{'?a': 10, '?b': 20}
>>> match_variable("?c", 30, bindings)
{'?a': 10, '?b': 20, '?c': 30}
>>> match_variable("?c", 30, bindings)
{'?a': 10, '?b': 20, '?c': 30}
>>> match_variable("?b", 40, bindings)
False
>>> match_variable("?d", 40, bindings)
{'?a': 10, '?b': 20, '?c': 30, '?d': 40}
```

3) contains_tokens : Checks if pattern list is not empty list

```
def contains_tokens(pattern):
    """Test if pattern is a list of subpatterns."""
    return type(pattern) is list and len(pattern) > 0
```

```
# >>> contains_tokens([1,2,3])
# True
# >>> contains_tokens([])
# False
# >>> contains_tokens('a')
# False
```

4) match_pattern : main pattern matching function

```
def match_pattern(pattern, input, bindings=None):
    if bindings is False:
        return False
    if pattern == input:
        return bindings
    bindings = bindings or {}
    if is_variable(pattern):
        var = pattern[1:]
        return match_variable(var, [input], bindings)
    elif contains_tokens(pattern) and contains_tokens(input):
        return match_pattern(pattern[1:], input[1:],
                              match_pattern(pattern[0], input[0], bindings))
    else:
        return False
```

```
>>> match_pattern(['a','?b','?c'], ['a','b','c'])
P= ['a', '?b', '?c'] I= ['a', 'b', 'c'] B = None
                                P= a I= a B = {}
P= ['?b', '?c'] I= ['b', 'c'] B = {}
                                P= ?b I= b B = {}
P= ['?c'] I= ['c'] B = {'b': ['b']}
                                P= ?c I= c B = {'b': ['b']}
P= [] I= [] B = {'b': ['b'], 'c': ['c']}
{'b': ['b'], 'c': ['c']}
```

```
>>> match_pattern(['a', '?d', '?e'], ['a', 'b', 'c'])
P= ['a', '?d', '?e'] I= ['a', 'b', 'c'] B = None
                                P= a I= a B = {}
P= ['?d', '?e'] I= ['b', 'c'] B = {}
                                P= ?d I= b B = {}
P= ['?e'] I= ['c'] B = {'d': ['b']}
                                P= ?e I= c B = {'d': ['b']}
P= [] I= [] B = {'d': ['b'], 'e': ['c']}
{'d': ['b'], 'e': ['c']}
```

```
>>> match_pattern(['a', 'b', 'c'], ['a', 'b', 'd'])
P= ['a', 'b', 'c'] I= ['a', 'b', 'd'] B = None
                                P= a I= a B = {}
P= ['b', 'c'] I= ['b', 'd'] B = {}
                                P= b I= b B = {}
P= ['c'] I= ['d'] B = {}
                                P= c I= d B = {}
P= [] I= [] B = False
False
```

```
5) def match(pattern, input):
    return match_pattern(pattern.split(), input.split())
```

```
print(match( '?a my name is ?*n', 'hello my name is ali'))
```

```
P= ['?a', 'my', 'name', 'is', '?n'] I= ['hello', 'my', 'name', 'is', 'ali'] B = None
```

```
P= ?a I= hello B = {}
```

```
P= ['my', 'name', 'is', '?n'] I= ['my', 'name', 'is', 'ali'] B = {'a': ['hello']}
```

```
P= my I= my B = {'a': ['hello']}
```

```
P= ['name', 'is', '?n'] I= ['name', 'is', 'ali'] B = {'a': ['hello']}
```

```
P= name I= name B = {'a': ['hello']}
```

```
P= ['is', '?n'] I= ['is', 'ali'] B = {'a': ['hello']}
```

```
P= is I= is B = {'a': ['hello']}
```

```
P= ['?n'] I= ['ali'] B = {'a': ['hello']}
```

```
P= ?n I= ali B = {'a': ['hello']}
```

```
P= [] I= [] B = {'a': ['hello'], 'n': ['ali']}
```

```
{'a': ['hello'], 'n': ['ali']}
```